

MANUAL DO SITE.

Como editar cada texto, cada cor, cada elemento.
Como ver o site rodando. Como conectar um domínio próprio.
Como publicar.

ÍNDICE.

01	COMO VER O SITE	p. 03
02	CONECTAR UM DOMÍNIO PRÓPRIO	p. 04
03	ESTRUTURA DO PROJETO	p. 06
04	EDITAR VIA CMS (KEYSTATIC)	p. 07
05	EDITAR TEXTOS DA INTERFACE (PT/EN)	p. 09
06	EDITAR POSTS, PROJETOS, PUBLICAÇÕES	p. 10
07	TROCAR CORES, FONTES, IDENTIDADE	p. 12
08	TROCAR A MARCA (NS//26) E O NOME	p. 13
09	ADICIONAR FOTO/RETRATO E PDF DO CV	p. 14
10	PUBLICAR MUDANÇAS	p. 15
11	RESOLUÇÃO DE PROBLEMAS	p. 16

VER O SITE.

Tem três formas. Da menor pra maior fricção.

A. DIRETO NO GITHUB PAGES (JÁ ESTÁ NO AR)

O repositório está configurado pra publicar automaticamente em todo push pra `main`. URL pública:

```
https://kaluanbernardo.github.io/personal-website/
```

Pra acompanhar o status do deploy, abre **Actions** no repo do GitHub e procura a última run "Deploy". Se for primeira execução, garanta em **Settings → Pages → Source: GitHub Actions**.

B. LOCAL EM MODO DEV (COM HOT-RELOAD)

No terminal, dentro da pasta do projeto:

```
# instala dependências (só na primeira vez)
npm install

# sobe o servidor de desenvolvimento
npm run dev
```

Abre no navegador: `http://localhost:4321/`

Esse modo recarrega sozinho a cada arquivo salvo. **Ideal pra editar conteúdo.**

C. LOCAL IMITANDO PRODUÇÃO

```
npm run build
npm run preview
```

Abre `http://localhost:4321/personal-website/`. Esse usa exatamente a mesma base de URL do GitHub Pages – útil pra testar links antes de subir.

► ATALHO DIÁRIO

Pra editar conteúdo, o fluxo é: roda `npm run dev`, edita o arquivo, salva, vê no navegador. Quando estiver bom, comita e dá push – o deploy é automático.

DOMÍNIO PRÓPRIO.

5 passos. Demora ~1h ativa + até 24h de propagação DNS.

1. COMPRAR O DOMÍNIO

- **.com.br**: registro.br (mais barato, ~R\$ 40/ano).
- **.com** / **.me** / **.dev**: Cloudflare Registrar, Namecheap, Porkbun (preço de custo, sem upsell).

2. CONFIGURAR O DNS

No painel da onde você comprou (ou no Cloudflare DNS, se levar pra lá), aponte para o GitHub Pages:

Para o domínio raiz (`seudominio.com`) – adicione 4 registros A:

```
A @ 185.199.108.153
A @ 185.199.109.153
A @ 185.199.110.153
A @ 185.199.111.153
```

Para o www (`www.seudominio.com`) – um CNAME:

```
CNAME www kaluanbernardo.github.io.
```

3. ADICIONAR ARQUIVO CNAME NO REPO

Crie o arquivo `public/CNAME` com uma única linha – o seu domínio:

```
seudominio.com
```

Isso garante que o GitHub Pages saiba qual domínio responder.

4. AJUSTAR A CONFIG DO ASTRO

Em `astro.config.mjs`, troque o bloco `base` pra ficar na raiz:

```
// antes:
base: isDev ? '/' : '/personal-website/',

// depois:
```

```
base: '/',  
site: 'https://seudominio.com',
```

Sem isso, os links continuariam apontando pra `/personal-website/...`.

5. CONFIGURAR NAS SETTINGS DO GITHUB

No GitHub: **Settings** → **Pages** → **Custom domain** → digite `seudominio.com` → salvar.

Aguarde alguns minutos e marque **Enforce HTTPS**.

★ RECOMENDAÇÃO

Se possível, mude o DNS pra Cloudflare (gratuito) – fica mais rápido, ganha proteção contra ataques e os relatórios de tráfego. Você só muda os `nameservers` no registrador e configura tudo no painel deles.

DNS pode demorar até 24h pra propagar mundialmente. Pra testar, use `dig seudominio.com` ou `dnschecker.org`.

ESTRUTURA.

Os arquivos que importam, do mais editável pro mais técnico.

```
personal-website/
├─ src/content/          ★ conteúdo do blog/projetos/publicações
│  ├─ posts/            → um arquivo .mdoc por post
│  ├─ projects/         → um arquivo .mdoc por projeto
│  └─ pubs/             → publicações acadêmicas
├─ src/i18n/ui.ts        ★ TODOS os textos da interface (PT + EN)
├─ src/styles/global.css ★ cores, fontes, look-and-feel
├─ src/components/       peças reutilizáveis
│  ├─ Nav.astro          → menu de navegação
│  ├─ Footer.astro       → rodapé
│  ├─ Marquee.astro      → a faixa rolando no topo
│  ├─ StatusBar.astro    → barra do rodapé com relógio
│  └─ pages/             → corpo de cada página
│     ├─ HomePage.astro
│     ├─ SobrePage.astro
│     ├─ CVPPage.astro
│     ├─ AcademicoPage.astro
│     ├─ BlogIndexPage.astro
│     ├─ BlogPostPage.astro
│     ├─ ProjetosPage.astro
│     └─ ContatoPage.astro
├─ src/pages/            rotas (URLs)
│  ├─ index.astro        → /
│  ├─ sobre.astro        → /sobre
│  ├─ cv.astro           → /cv
│  ├─ ...
│  └─ en/                → versão em inglês: /en/sobre, /en/cv...
├─ public/               arquivos servidos como estão
│  ├─ img/portrait.jpg   → sua foto (opcional)
│  ├─ cv-pt.pdf          → PDF do CV em PT
│  ├─ cv-en.pdf          → PDF do CV em EN
│  └─ CNAME              → domínio próprio (se for usar)
└─ keystatic.config.ts   painel admin do CMS
```

EDITAR VIA CMS.

O modo mais fácil. Não precisa abrir nenhum arquivo de texto.

ACESSAR O PAINEL

Com o servidor de desenvolvimento rodando (`npm run dev`), abre:

```
http://localhost:4321/keystatic
```

Vai aparecer a tela de login. Selecione "**Local**" – você está editando os arquivos diretamente.

⚠ IMPORTANTE

O Keystatic só está disponível em **modo dev** (não vai pro site público). É uma interface pra editar arquivos locais. Você ainda precisa **comitar** e **dar push** pra publicar.

O QUE DÁ PRA EDITAR PELO PAINEL

- **Posts** (do blog) – título, descrição, data, tag, idioma, marcar como rascunho/destaque, conteúdo em rich text.
- **Projetos** – título, tipo, ano, veículo/local, URL externa, descrição.
- **Publicações acadêmicas** – título, venue, ano, tipo (artigo/capítulo/livro/tese/dissertação), URL.

FLUXO: CRIAR UM NOVO POST

1. No painel, clique em **Posts → "+ Add"**.
2. Preenche os campos (título, descrição, data, tag, tempo de leitura).
3. Marca o idioma (PT ou EN).
4. Escreve o corpo no editor.
5. Clica em **Save**.
6. O Keystatic gera o arquivo `.mdoc` em `src/content/posts/`.
7. Comita e dá push pra publicar:

```
git add src/content/posts/  
git commit -m "novo post: título"  
git push
```


EDITAR UM POST EXISTENTE

No painel, clica no post na lista, faz a edição, salva. Comita e push.

DELETAR

No painel, abre o item e clica em **Delete**. O arquivo é removido.
Comita e push.

TEXTOS UI.

O Keystatic gerencia conteúdo. Pros textos do menu, hero, botões, etc., tem um arquivo central.

Tudo está em `src/il8n/ui.ts`. É um único arquivo com dois objetos: `pt` e `en`. Tem mais ou menos 530 linhas, mas é só prosa.

EXEMPLO: TROCAR A CHAMADA DA HOME

```
// src/il8n/ui.ts – bloco "home" do português
home: {
  eyebrow: '- site pessoal · ed. 2026',
  title: { line1: 'Nome', line2: 'Sobrenome.' },
  intro:
    'Product manager em educação, jornalista freelancer e ...',
  ctaBlog: 'Ler o blog →',
  ctaCV: 'Baixar CV',
  ...
}
```

TROCAR O NOME EM TODO LUGAR

Procura por `'nome sobrenome'` (PT) e `'first last'` (EN) no arquivo e substitui. Esses entram automaticamente no header, no monograma, na tag `NS//26`, etc.

MENSAGENS DA NEWSLETTER, DO BLOG, DA CONTATO

Dentro de `ui.ts` tem um bloco por seção:

- `home` – hero, cards, "do blog"
- `sobre` – eyebrow, título, parágrafos, "agora"
- `cv` – experiência, skills
- `academico` – linhas de pesquisa, formação
- `blog` – título, filtros, copy da newsletter
- `projetos` – título, subtítulo, filtros
- `contato` – texto, e-mail, redes sociais
- `footer` – texto do rodapé

Abre o arquivo num editor de texto, encontra o bloco da seção, troca a string entre aspas. Salva. Se o `npm run dev` estiver rodando, recarrega sozinho.

Versão em inglês fica logo abaixo, dentro do mesmo arquivo, no objeto `en`. Mantém os dois sincronizados.

POSTS · PROJETOS · PUBLS.

Pode editar pelo Keystatic ou direto nos arquivos.
Aqui o esquema dos arquivos.

POSTS DO BLOG

Pasta: `src/content/posts/` · um arquivo `.mdoc` por post. Nome do arquivo vira o slug da URL.

```
---
title: "Sobre escrever em três modos"
description: "Como manter especificação técnica..."
pubDate: 2026-04-28
readingTime: 8
tag: "ensaio"          # ensaio | produto | jornalismo | leitura | notas |
                           academia
featured: true          # aparece em destaque na lista
draft: false            # true = não publica
lang: "pt"             # pt | en
---

## Subtítulo

Texto do post em markdown. Pode usar negrito, itálico,
links em `[texto](url)`, listas, citações com `>`...
```

PROJETOS

Pasta: `src/content/projects/`

```
---
title: "Reportagem · plataforma X"
kind: "jornalismo"      # jornalismo | produto | academia | palestra |
                           pessoal
place: "Veículo"
year: 2025
url: "https://link.externo" # opcional
thumb: "img/projetos/foo.jpg" # opcional, dentro de public/
lang: "pt"
---

Descrição opcional do projeto.
```

PUBLICAÇÕES ACADÊMICAS

Pasta: `src/content/pubs/`

```
---
title: "Métricas de atenção em plataformas educacionais"
venue: "Revista Brasileira de Comunicação · v.42, n.2"
year: 2024
kind: "artigo" # artigo | capítulo | livro | tese | dissertação
url: "https://doi.org/..."
lang: "pt"
---
```

VERSÃO EM INGLÊS

Pra cada post/projeto/pub em PT, crie outro arquivo com o mesmo conteúdo traduzido e `lang: "en"`. Sugestão: termine o nome do arquivo com `-en` pra ficar fácil de identificar.

CORES ■ FONTES.

Tudo controlado por variáveis CSS. Trocar uma linha muda o site inteiro.

ONDE MEXER

Arquivo: `src/styles/global.css` – bloco `:root { ... }` no topo.

PALETA NEON (ATUAL)

```
:root {
  --bg:    #0a0a0a; /* fundo (modo escuro) */
  --fg:    #f5f5f0; /* texto */
  --paper: #ebe7d8; /* fundo modo claro/zine */

  --hot:   #ff2d55; /* magenta · usado na seção contato */
  --cyan:  #00f0ff; /* ciano · seção sobre */
  --acid:  #c4ff00; /* verde ácido · home + acadêmico */
  --ylw:   #ffe600; /* amarelo · cv */
  --orng:  #ff6a00; /* laranja · projetos */
  --vio:   #b400ff; /* roxo · blog */
}
```

TROCAR A COR DE UMA SEÇÃO

Mais embaixo no mesmo arquivo, tem o mapa de tema por página:

```
.theme-home    { --accent: var(--acid); }
.theme-sobre   { --accent: var(--cyan); }
.theme-cv      { --accent: var(--ylw); }
.theme-academ  { --accent: var(--acid); }
.theme-blog    { --accent: var(--vio); }
.theme-projetos { --accent: var(--orng); }
.theme-contato { --accent: var(--hot); }
```

Trocar `var(--cyan)` por `var(--orng)` em `.theme-sobre`, por exemplo, vai pintar a página Sobre toda em laranja.

FONTES

Carregadas do Google Fonts no topo do `global.css`. Os papéis:

VARIÁVEL	FONTE	ONDE APARECE
--bungee	Bungee	headlines gigantes

--bungee-s	Bungee Shade	variação com sombra
--mono	Departure Mono → Space Mono (fallback)	texto base, navegação, marquee
--pixel	VT323	números, terminal vibe
--tiny	Press Start 2P	etiquetas micro
--serif	Times New Roman	corpo de texto longo (sobre, post)

MARCA **NS//26**.

A tag rosa do canto superior esquerdo. Onde mudar o texto e o nome.

TROCAR O TEXTO DA TAG

Arquivo: `src/components/Monogram.astro`

```
const { letter = 'NS//26' } = Astro.props;
```

Troca `'NS//26'` pelo que quiser (ex.: `'KB//26'`, `'XX'`, suas iniciais...).

TROCAR O NOME AO LADO DA TAG

Arquivo: `src/i18n/ui.ts` – campo `brand`:

```
pt: { brand: 'nome sobrenome', ... }  
en: { brand: 'first last', ... }
```

Esse `brand` também alimenta o título do hero da home (que mostra **NOMESOBRENOME** em letras gigantes).

TROCAR O TÍTULO DO NAVEGADOR

Por página, abre o arquivo correspondente em `src/pages/` e ajuste o `title`:

```
<Page title="Nome Sobrenome" description="Site pessoal..." ...>
```

TROCAR O FAVICON

O favicon é gerado inline em `src/layouts/Base.astro` (SVG com letras "NS"). Substitua o atributo `href` da tag `<link rel="icon">` por:

- outro SVG inline (mesmo padrão), ou
- um arquivo: salva em `public/favicon.png` e usa `href="/favicon.png"`.

Tag colorida → `Monogram.astro` . Nome → `ui.ts` (campo `brand`). Título da aba → `src/pages/<rota>.astro` . Favicon → `Base.astro` .

FOTOS ■ PDFS.

Tudo que é binário (imagem, PDF, fonte) vive na pasta `public/`.

FOTO / RETRATO

A página Sobre tenta carregar `public/img/portrait.jpg` automaticamente. Se o arquivo existir, aparece. Se não existir, mostra um placeholder.

```
# crie a pasta se ainda não existe
mkdir -p public/img

# copie sua foto pra lá com o nome exato
cp ~/Downloads/minha-foto.jpg public/img/portrait.jpg
```

Recomendado: 4x5 (vertical), pelo menos 800x1000px, `.jpg` ou `.png`.

PDF DO CURRÍCULO

O botão "Baixar CV" da home aponta pra:

- `public/cv-pt.pdf` – versão portuguesa
- `public/cv-en.pdf` – versão inglesa

```
cp meu-cv.pdf public/cv-pt.pdf
cp my-resume.pdf public/cv-en.pdf
```

THUMBNAILS DE PROJETOS

Hoje os cards de projeto mostram um placeholder (linhas diagonais com texto "thumb · 4x3"). Pra usar fotos reais:

1. Salve a imagem em `public/img/projetos/foo.jpg` (recomendado 4x3, 600x450px).
2. Edite o projeto correspondente e adicione `thumb:`
`"img/projetos/foo.jpg"`.
3. Edite `src/components/pages/ProjetosPage.astro` dentro de `.thumb` pra renderizar `<img src=...>` quando `thumb` estiver presente. (Hoje é só placeholder – se quiser, peço pra eu adicionar.)

OUTRAS IMAGENS EM POSTS

Salve em `public/img/posts/foo.jpg` e referencie no post:

```
![alt da imagem](/personal-website/img/posts/foo.jpg)
```

Atenção: se mudar pra domínio próprio (sem `/personal-website/`), use só `/img/posts/foo.jpg`.

PUBLICAR.

Tudo é git. Push pro `main` dispara deploy automático.

FLUXO PADRÃO

```
# 1. ver o que mudou
git status

# 2. adicionar os arquivos
git add src/content/posts/novo-post.mdoc
# ou tudo de uma vez:
git add -A

# 3. comitar com uma mensagem clara
git commit -m "Novo post: economia da atenção"

# 4. push pro GitHub
git push
```

Em ~1-2 minutos o GitHub Actions compila e publica. Acompanhe em **Actions** no repo.

TRABALHANDO EM UMA BRANCH

Pra mudanças maiores (ex.: trocar paleta de cores, redesenhar uma página), use uma branch:

```
git checkout -b experimento-cores
# edita o que quiser
git add -A && git commit -m "experimento: paleta verde"
git push -u origin experimento-cores
```

Abra um Pull Request no GitHub. Quando estiver bom, clica em **Merge**. O deploy só roda no `main`.

⚠ CUIDADO

Antes de comitar, rode `npm run build`. Se passar sem erros, está tudo certo. Se quebrar, o deploy também vai quebrar.

REVERTER UMA MUDANÇA

Se publicou algo que não queria:

```
git revert HEAD      # gera um novo commit que desfaz o último  
git push
```

Não use `git reset --hard` em commits já publicados – você apaga histórico.

QUANDO QUEBRA.

Os problemas mais comuns e como resolver.

"NPM RUN DEV" FALHA COM 'ASTRO: NOT FOUND'

As dependências não foram instaladas. Roda:

```
npm install
```

PÁGINA EM BRANCO / 404 AO ABRIR LOCAL

Em modo dev, o site roda em `http://localhost:4321/` (sem o `/personal-website/`). Se você abriu `http://localhost:4321/personal-website/`, vai dar 404. Tira o `/personal-website` da URL.

MUDEI O TEXTO E NADA APARECEU

- Confirme que salvou o arquivo.
- Confirma que `npm run dev` ainda está rodando.
- Recarrega a página com `Ctrl+Shift+R` (force reload).

BUILD FALHA NO GITHUB ACTIONS

Abre **Actions** no repo, clica na run com X vermelho, lê o erro.

- **Erro de tipo:** rode `npx astro check` local pra ver onde.
- **Erro de schema:** provavelmente um campo obrigatório faltando num `.mdoc` (ex.: `readingTime`).
- **Erro de import:** provavelmente um arquivo deletado mas ainda referenciado.

HTTPS DO DOMÍNIO PRÓPRIO NÃO HABILITA

Esperar 1-2h depois de configurar. Se persistir, em **Settings** → **Pages**, clica em **Remove** no domínio, espera 5min, adiciona de novo.

KEYSTATIC MOSTRA "CANNOT CONNECT"

Roda `npm run dev` antes de abrir `/keystatic`. O CMS é parte do servidor de dev.

QUERO MUDAR O ANO "EDIÇÃO 04 · 2026"

Arquivo: `src/components/pages/HomePage.astro` – procura por `EDIÇÃO 04` e edita.

PARA TUDO O RESTO

Em qualquer dúvida específica ("como mudo X?"), me pergunte com a captura de tela ou o nome da seção. Posso aplicar a mudança e gerar um diff.